Quality assurance Golfit

1. Approach

Our QA strategy follows a multi-layered approach:

Preventive QA: Avoid bugs using coding standards and documentation

Detective QA: Identify bugs through testing

Corrective QA: Fix bugs using structured bug tracking

Continuous Improvement: Improve processes using feedback and metrics

2. **Goals / Dimensions**

Dimension	Description	Measurement method
Functionality	Game features work properly	Functional test cases
Reliability	System operates without failures	Stability testing
Security	Protection against exploits	Exploits testing
Maintainability	Code is easy to modify	Code analysis tools
Scalability	Ability to handle multiple players	Performance testing
Usability	The game is easy to understand and play	Playtesting

3. Constructive QA

Coding Standards: Consistent coding style to improve readability and reduce errors.

Documentation: Use of structured comments (e.g., JavaDoc) to describe code behavior.

Exception Handling: Proper handling of runtime errors to ensure system stability.

Logging: Recording system events and errors to support debugging. These measures help reduce the number of defects introduced during development.

4. Analytical QA

Code Reviews: Developers review each other's code to detect issues early.

Testing: Systematic execution of test cases to validate functionality.

Debugging: Structured process to reproduce, isolate, and fix errors.

5. Testing strategy

Test Levels

Unit Testing: Testing individual components such as game logic functions.

Integration Testing: Verifying interaction between components (e.g., client-server communication).

System / Acceptance Testing: Testing the complete system through gameplay scenarios.

Test Types

Functional Testing: Ensures all features work as intended.

Performance Testing: Measures system performance (e.g., frame rate, response time).

Load Testing: Evaluates system behavior under multiple users or sessions.

Usability Testing: Assesses user experience and interface clarity.

Regression Testing: Ensures new changes do not break existing functionality

6. Metrics

Core Metrics:

- Test coverage percentage (code and requirements)
- Server response time under various loads
- Client frame rate on target hardware

Quality Process Metrics:

- Time spent on bug fixing vs. new feature development
- Number of builds rejected due to quality issues
- Code review effectiveness (issues found per review)

7. **First metrics report**

Code quality metrics

Metric	Current value	Target Value	Goal achieved
Unit test coverage	0	>=80%	x
Cyclomatic complexity (avg)	2.19	<=5	✓
Documentation issues	100 (	<=50	x
Static Analysis Issues	59	<=5 (0 high prio)	x

Performance Metrics

Server response time (avg)			
Client Frame Rate (min)			
Network Bandwidth			

Process Metrics

Defects Found in Review			
Defects Found in Testing			
Client Frame Rate (min)			
Test Automation Time	16.756s	<=2min	✓
Build Success Rate	100%	100%	✓

First key findings :

Automation time and build success rate are good, the code isn't too deeply nested on average. The documentation as well as the source code need improvement

8. Milestones

Until Milestone 4 :

- Implement Jacoco and MetricsReloaded for first metrics report
- Create QA concept

Until Milestone 5 :

- Create mock objects for network and dependencies
- Implement parameterized tests for boundary
- Build automated tests for client-server communication
- Create simulation framework for multi-client scenarios
- Implement network condition simulators (latency, packet loss) Performance Test Suite
- Create client performance benchmarking tool

Until Milestone 6 :

- Implement server performance monitoring
- Define structured playtest scenarios to validate specific aspects
- Identify bottlenecks and improvement opportunities